Memoryless Timing Is a Regime, Not a Constant

First-passage structure of rupture times in Coherence–Rupture–Regeneration

Alexander Sabine · Active Inference Institute · alexander@activeinference.institute

October 2026

Abstract. Coherence–Rupture–Regeneration (CRR) defines a rupture, or cut, as the first time a system's net forward travel on a statistical carrier reaches a half-turn of Fisher–Rao arc. I ask what this rule implies about the timing of successive cuts, and in particular about memoryless (exponential) timing, the signature of processes such as radioactive decay.

Approach. Combining the cut rule with an overdamped Langevin law on a circular carrier, I show that inter-cut timing is set by the added dynamics, not by geometry, and moves continuously between three regimes:

1. **Directed accumulation:** nearly regular timing, with coefficient of variation (CV) ≈ 0.11.
2. **Balanced drift and diffusion:** inverse-Gaussian timing with CV ≈ 1.
3. **Escape from a potential well:** exponential, memoryless timing.

Results. Two observations follow:

- A CV of 1 does not identify memorylessness. The balanced regime has CV ≈ 1, yet a matched exponential is rejected on every seed (KS ≥ 0.117).
- In the escape regime the path is almost entirely surplus, about 270 half-turns of travel per half-turn of progress. That surplus correlates perfectly with elapsed time ($r > 0.9999$), so it carries no information about when the next cut will come.

Memorylessness is therefore a dynamical regime of CRR, not a geometric constant, and it is the regime in which a system's history is uninformative about its future.

Implication for method. Memoryless processes are a natural must-fail control for CRR history statistics.

How it was tested. All predictions were preregistered, frozen and externally timestamped before confirmation on ten independent seeds.

Keywords: Coherence–Rupture–Regeneration; first passage; inverse Gaussian; Kramers escape; memorylessness; surplus; negative control.

1. Introduction

Many persisting systems alternate between accumulation and abrupt transition: neurons fire, cells divide, organisms grow in spurts, nuclei decay. The statistics of the intervals between transitions are a basic fingerprint of the underlying process.

- **Periodic processes** have small coefficients of variation.
- **Renewal processes driven by diffusion** have inverse-Gaussian intervals.
- **Memoryless processes** have exponential intervals.

For memoryless processes the expected remaining wait does not depend on how long one has already waited.

What CRR offers here. CRR describes persistence through three operations:

- **coherence**, the accumulation of distinguishable structure, measured as Fisher–Rao arc on a statistical carrier;
- **rupture**, a cut at a threshold;
- **regeneration**, the reconstruction of the next occasion from completed ones.

It therefore makes the timing of cuts a natural object of study. An earlier formulation of CRR tied cut timing directly to carrier geometry, predicting a fixed CV from the threshold parameter Ω and treating the unit CV of the exponential distribution as a product of geometric constants. In the current formulation the threshold is a single half-turn for every carrier, and physical dynamics are added to the framework rather than derived from it.

This paper works out what the current formulation implies for cut timing. It answers two questions:

1. Which dynamical regimes produce memoryless cut times?
2. Is a unit CV sufficient evidence of memorylessness?

2. Framework

2.1 Carrier, coherence and surplus

The setting is the current CRR core:

- **Carrier.** An intrinsic circle of probability distributions with Fisher metric $g = 1$ and circumference $\ell = 2\pi$, so the half-turn is $a = \pi$.
- **Coherence.** C is the accumulated Fisher–Rao arc of the path.
- **Chord.** C^* is the geodesic distance between an occasion's start and end.
- **Surplus.** $S = C - C^*$, the travel in excess of the chord.

On a completed half-turn with backward travel B, the surplus is exactly $S = 2B$, and a monotone half-turn has $S = 0$.

2.2 The cut

A cut occurs at the first passage of net forward travel through a half-turn,

$$t_{(n+1)} = \inf\{\, t > t_n : s(t) - s(t_n) = a \,\},$$

with threshold parameter $\Omega = 1/a$, the same for every carrier.

2.3 Dynamics and regeneration

The framework fixes the geometry of a cut but not the physical law that moves a system along its carrier. Following the core's treatment of physical laws as constitutive additions, I take overdamped motion with unit friction, a tilt τ, a periodic potential $U(s) = A \cos 2s$ (one barrier per half-turn) and noise intensity D:

$$ds = (\tau - U'(s))\, dt + \sqrt{(2D)}\, dW.$$

Each occasion regenerates at the bottom of a potential well, so successive cut intervals are independent and identically distributed (a renewal process).

3. Methods

3.1 Regimes

Regime	Tilt τ	Barrier A	Noise D	Character
Directed accumulation	1	0	0.02	drift-dominated
Balanced drift and diffusion	1	0	$\pi/2$	Péclet number $\tau a/D = 2$
Escape from a well	0.3	1	0.3	barrier-dominated (Kramers regime)

3.2 Measures

- **Exact mean.** The exact mean first-passage time is $T = (1/D) \int$ from s_0 to s_0+a of $dy\, e^{V(y)/D} \int$ from $-\infty$ to y of $dz\, e^{-V(z)/D}$, with $V = U - \tau s$.
- **Shape tests.** Kolmogorov–Smirnov (KS) distances to an exponential with the same mean, and to an inverse Gaussian with the same mean and variance.
- **Mean residual life.** The expected remaining wait at the median, divided by the mean. It equals 1 at every age if and only if the timing is memoryless.
- **Surplus.** S/a and its correlation with the interval length.

3.3 Simulation and preregistration

Simulation. Euler–Maruyama integration with step 0.002, 4,000 occasions per seed, and surplus sampled at a resolution of 0.05.

Preregistration.

- **Thresholds.** The decision thresholds were set on development seeds 0–2 and frozen together with the code and scorer (study `MEMORYLESS-CHECK-2026-10-27`; FREEZE sha256 `056a30cc…217a`).
- **Anchor.** The freeze was anchored by a pushed commit (`52c8f75`) and an OpenTimestamps stamp before ten confirmation seeds (1000–1009) were generated.
- **Reproducibility.** Two independent reruns reproduced all outputs byte for byte.

- **Instrument controls**, scored before the freeze: true exponential samples meet the memoryless criteria, and inverse-Gaussian samples with CV = 1 do not.

Preregistered predictions:

1. Escape-regime intervals are memoryless: KS distance to the exponential within the 5% critical value (0.0215) on at least 7 of 10 seeds, and residual-life ratio in [0.95, 1.10].
2. Balanced-regime intervals have CV in [0.95, 1.05] but reject the exponential (KS > 0.05) on every seed.
3. Escape-regime surplus exceeds 50 half-turns and correlates with interval length at r ≥ 0.99.
4. Directed intervals have CV ≤ 0.2.

4. Results

All preregistered predictions held on all ten confirmation seeds. Simulated means agreed with the exact first-passage formula to within 1.3%.

Table 1. Confirmation seeds 1000–1009, n = 4,000 intervals per seed; ranges across seeds.

Regime	Exact mean	Simulated mean	CV	KS vs exponential	KS vs inverse Gaussian	Residual life / mean	Surplus / a
Directed	3.14	3.15	0.111–0.114	0.525–0.529	0.007–0.016	0.09–0.10	0.1
Balanced	3.14	3.18	**0.963–1.035**	**0.117–0.125**	0.007–0.020	0.91–0.95	5–6
Escape	323.5	324.6	**1.021–1.076**	**0.008–0.021**	0.111–0.125	**1.01–1.05**	268–282

Three regimes, three distributions.

- **Directed accumulation** gives near-regular intervals (CV ≈ 0.11) that are well described by an inverse Gaussian.
- **Balanced drift and diffusion** gives inverse-Gaussian intervals with CV ≈ 1: the exponential is decisively rejected, and the residual-life ratio rises with age.
- **Escape from a well** gives exponential intervals: the exponential fits, the inverse Gaussian is rejected, and the expected remaining wait is independent of the time already waited (Figure 1).

Surplus in the escape regime. The path travels 268–282 half-turns in total per half-turn of net progress, and surplus correlates with interval length at r > 0.9999 on every seed. Nearly all of the path is wandering within the well, and its extent records elapsed time only.

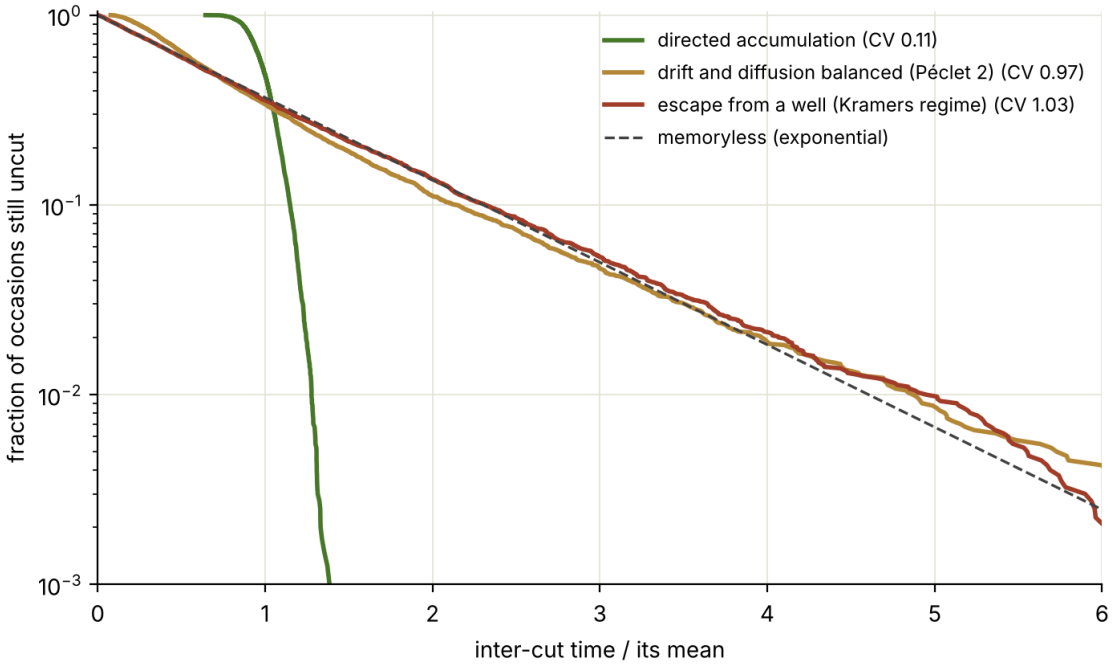


Figure 1. Fraction of occasions not yet cut against interval length divided by its mean, on a log scale (confirmation seed 1000). Dashed line: the exponential. The escape regime follows it. The balanced regime has the same CV but starts flat: early cuts are rarer than memorylessness allows. Directed accumulation falls off sharply around the mean.

5. Discussion

5.1 Memorylessness is a regime of CRR

Under the current cut rule, one geometry (a half-turn of Fisher–Rao travel) supports regular, inverse-Gaussian and exponential timing, depending only on the dynamics that move the system along its carrier.

Memoryless timing arises in one regime: when the cut is dominated by a single rare escape from a trapped state rather than by steady accumulation. This replaces the earlier formulation's view that cut variability is fixed by carrier geometry, including the reading of CV = 1 as a product of geometric constants. In the current framework, variability is a property of the system's dynamics, and geometry fixes only where the cut lies.

5.2 A unit CV is not evidence of memorylessness

The balanced regime has the same CV as the escape regime and a different distribution. Inferring memorylessness, or any process class, from CV alone is therefore unsafe. Distribution shape, hazard or residual life must be examined.

5.3 History without information

CRR distinguishes the net change of an occasion (the chord) from the path actually travelled (the coherence). The difference is the surplus. In learning systems, surplus is the part of experience that may be written into what comes next.

The escape regime is the opposite limit:

- surplus is maximal, about 270 times the chord;
- it is entirely uninformative about the timing of the next cut, because the process forgets its path at every instant.

Memoryless timing is thus the regime in which a system has the most history and the least use for it.

5.4 A negative control for CRR methods

Radioactive decay is the physical archetype of escape-dominated timing. Gamow explained alpha decay as escape from a nuclear well by tunnelling.

Any CRR statistic that claims to carry predictive history, whether surplus, cut counts or an occasion clock, should therefore show no predictive power on memoryless data. I propose such processes as a standing must-fail control for CRR methods: a positive result on them indicates leakage, not structure.

5.5 Relation to classical theory

The three regimes correspond to well-established results:

- **Inverse-Gaussian first passage** of drifted Brownian motion (Schrödinger 1915).
- **Exponential escape over a barrier** at weak noise (Kramers 1940).
- **The exact passage-time integrals** for tilted periodic potentials (e.g. Reimann et al. 2001).

CRR's contribution is the identification of the cut with a first passage of Fisher–Rao travel, together with the surplus accounting that makes the informational character of each regime explicit.

5.6 Limitations

- **Surplus measured on a noisy path depends on sampling resolution.** In the escape regime it was 441, 272 and 121 half-turns at steps of 0.02, 0.05 and 0.2. The qualitative pattern is robust, but absolute values require the calibrated resolution σ that the CRR core specifies.
- **One potential shape and a renewal rule were examined.** Regeneration that carries information between occasions would make intervals dependent.
- **The model is classical and overdamped.** The link to radioactive decay is structural.
- **The evidential status is preliminary.** The study was preregistered and confirmed on fresh seeds, but without an owner-signed tag before the run and without an independent operator. Its evidential ceiling under the Phoenix protocol is PASS-0.

6. Conclusion

Under the current form of CRR, the timing of cuts is not fixed by geometry. A single cut rule supports regular, inverse-Gaussian and memoryless timing, according to the dynamics that drive accumulation.

Memorylessness occurs when cuts are dominated by escape from a trapped state. A unit coefficient of variation alone does not reveal it. In that regime, surplus is maximal and uninformative. Memoryless processes therefore mark the boundary of CRR's history-based explanations, and they serve as a natural negative control for them.

Data and code availability

All code is listed in full in the Appendix and is held in the Phoenix repository:

- the study: `Phoenix_Egg/studies/MEMORYLESS-CHECK-2026-10-27/`;
- the freeze, anchor, exposure and rerun receipts: `Phoenix_Egg/receipts/MEMORYLESS-CHECK-2026-10-27/`.

Requirements: Python 3.13, numpy 2.5.3 and scipy 1.18.1, plus matplotlib for the figure. `sh run.sh RUNS_DIR` reproduces Table 1 (about 6 minutes on 4 cores), and `python3 figure.py` reproduces Figure 1.

References

- Gamow, G. (1928). Zur Quantentheorie des Atomkernes. *Zeitschrift für Physik* 51, 204–212.
- Kramers, H. A. (1940). Brownian motion in a field of force and the diffusion model of chemical reactions. *Physica* 7, 284 ff.
- Reimann, P., Van den Broeck, C., Linke, H., Hänggi, P., Rubi, J. M. & Pérez-Madrid, A. (2001). Giant acceleration of free diffusion by use of tilted periodic potentials. *Physical Review Letters* 87, 010602.
- Schrödinger, E. (1915). Zur Theorie der Fall- und Steigversuche an Teilchen mit Brownscher Bewegung. *Physikalische Zeitschrift* 16, 289–295.

Appendix: complete code

Each file is reproduced verbatim from the frozen study directory.

`model.py`

```
        """Model for MEMORYLESS-CHECK-2026-10-27 (frozen). Inter-cut times under the current CRR
cut rule.

        Carrier: intrinsic circle, Fisher metric g = 1, circumference ell = 2*pi, half-turn a = pi
(CORE R1).
        Cut:     t_(n+1) = inf{ t > t_n : s_t - s_(t_n) = a }   (net forward travel; CORE R2).
        Law:     ds = (tau - U'(s)) dt + sqrt(2 D) dW,  U(s) = A cos(2 s)   (CORE R4, overdamped,
zeta = 1).
        Renewal: every occasion starts at the bottom of a well (s0 = pi/2 when A > 0, else 0).
        Surplus: S = 2 B, B = backward travel sampled every `res` time units (CORE R3 at a stated
resolution).
        """
        import numpy as np
        from scipy import stats, integrate

        a = np.pi
        REGIMES = {
            "directed":        dict(tau=1.0, A=0.0, D=0.02),
            "drift_diffusion": dict(tau=1.0, A=0.0, D=np.pi / 2),   # Peclet number tau*a/D = 2
            "escape":          dict(tau=0.3, A=1.0, D=0.30),        # Kramers regime: one barrier
per half-turn
        }
        DT, RES, TMAX = 2e-3, 0.05, 20000.0


        def start(p):
            return np.pi / 2 if p["A"] > 0 else 0.0


        def simulate(tau, A, D, n, seed, s0, dt=DT, res=RES, tmax=TMAX):
            rng = np.random.default_rng(seed)
            s = np.full(n, s0); done = np.zeros(n, bool); T = np.full(n, np.nan)
            last = s.copy(); back = np.zeros(n); every = max(1, int(round(res / dt))); k = 0
            while not done.all() and k * dt < tmax:
                live = ~done
                s[live] += (tau + 2 * A * np.sin(2 * s[live])) * dt + np.sqrt(2 * D * dt) *
rng.standard_normal(live.sum())
                k += 1
                if k % every == 0:
                    back += np.where(live, np.maximum(last - s, 0), 0); last = s.copy()
                hit = live & (s - s0 >= a)
                T[hit] = k * dt; done |= hit
            return T, 2 * back, float((~done).mean())


        def exact_mean(tau, A, D, s0):
            """T = (1/D) int_{s0}^{s0+a} dy e^{V(y)/D} int_{-inf}^{y} dz e^{-V(z)/D},  V = U - tau
s."""
            V = lambda x: A * np.cos(2 * x) - tau * x
            lo = s0 - 60 * D / max(tau, 1e-9) - 20
            inner = lambda y: integrate.quad(lambda z: np.exp(-(V(z) - V(y)) / D), lo, y,
limit=400)[0]
            return integrate.quad(inner, s0, s0 + a, limit=200)[0] / D


        def describe(T):
            """Distribution diagnostics used by the frozen scorer."""
            T = np.asarray(T, float); T = T[~np.isnan(T)]; m = T.mean()
            q = np.quantile(T, [0.25, 0.5, 0.75])
```

```
            lam = m ** 3 / T.var()
            return dict(n=int(T.size), mean=float(m), cv=float(T.std() / m),
                        mrl=[float((T[T > x] - x).mean() / m) for x in q],
                        ks_exp=float(stats.kstest(T, 'expon', args=(0, m)).statistic),
                        ks_ig=float(stats.kstest(T, stats.invgauss(m / lam,
scale=lam).cdf).statistic))
```

run_seeds.py

```
        """Run the three regimes on a list of seeds and write outcomes JSON (deterministic; values
rounded to 6 d.p.).
        Usage: python3 -I run_seeds.py OUT.json N SEED [SEED ...]"""
        import json, sys, pathlib
        sys.path.insert(0, str(pathlib.Path(__file__).resolve().parent))
        from multiprocessing import Pool
        import numpy as np
        from model import REGIMES, simulate, start, describe, exact_mean, a

        def one(job):
            name, seed, n = job; p = REGIMES[name]
            T, S, cens = simulate(**p, n=n, seed=seed, s0=start(p))
            d = describe(T); ok = ~np.isnan(T)
            d.update(seed=seed, censored=cens, surplus_over_a=float(np.mean(S[ok]) / a),
corr_S_T=float(np.corrcoef(S[ok], T[ok])[0, 1]))
            return name, d

        def rnd(x):
            if isinstance(x, float): return round(x, 6)
            if isinstance(x, list): return [rnd(v) for v in x]
            if isinstance(x, dict): return {k: rnd(v) for k, v in x.items()}
            return x

        if __name__ == "__main__":
            out, n, seeds = sys.argv[1], int(sys.argv[2]), [int(s) for s in sys.argv[3:]]
            jobs = [(name, s, n) for name in REGIMES for s in seeds]
            with Pool(4) as pool: res = pool.map(one, jobs)
            o = {name: {"params": REGIMES[name], "exact_mean": exact_mean(**REGIMES[name],
s0=start(REGIMES[name])),
                        "seeds": [d for nm, d in res if nm == name]} for name in REGIMES}
            pathlib.Path(out).write_text(json.dumps(rnd(o), indent=1, sort_keys=True) + "\n")
```

score.py

```
        """Frozen scorer for MEMORYLESS-CHECK-2026-10-27. Usage: python3 -I score.py outcomes.json
> verdict.json
        Rules fixed before the holdout seeds were generated (see PREREG.md). n = 4000 per seed, 10
seeds; KS 5% critical value 1.358/sqrt(n)."""
        import json, math, sys
        o = json.load(open(sys.argv[1])); crit = 1.358 / math.sqrt(4000)
        E, B, D = o["escape"]["seeds"], o["drift_diffusion"]["seeds"], o["directed"]["seeds"]
        med = lambda xs: sorted(xs)[len(xs) // 2] if len(xs) % 2 else 0.5 * (sorted(xs)[len(xs) //
2 - 1] + sorted(xs)[len(xs) // 2])
        rel = lambda reg: abs(sum(s["mean"] for s in o[reg]["seeds"]) / len(o[reg]["seeds"]) -
o[reg]["exact_mean"]) / o[reg]["exact_mean"]
        checks = {
          "V0_harness_exact_mean_within_5pct": all(rel(r) <= 0.05 for r in o) and all(s["censored"]
== 0 for r in o for s in o[r]["seeds"]) and all(s["n"] == 4000 for r in o for s in o[r]["seeds"]),
          "H1_escape_memoryless": med([s["ks_exp"] for s in E]) <= crit and sum(s["ks_exp"] <= crit
for s in E) >= 7
                                  and all(0.95 <= s["mrl"][1] <= 1.10 for s in E) and
all(s["ks_ig"] > 0.05 for s in E),
          "H2_cv1_not_memoryless": all(0.95 <= s["cv"] <= 1.05 for s in B) and all(s["ks_exp"] >
0.05 for s in B)
                                   and sum(s["ks_ig"] <= 0.04 for s in B) >= 9,
          "H3_escape_surplus_uninformative": all(s["corr_S_T"] >= 0.99 for s in E) and
```

```
all(s["surplus_over_a"] >= 50 for s in E),
          "H4_directed_not_memoryless": all(s["cv"] <= 0.2 for s in D) and all(s["ks_exp"] > 0.3
for s in D),
        }
        verdict = "VOID" if not checks["V0_harness_exact_mean_within_5pct"] else ("RESULT" if
all(checks.values()) else "FAIL")
        json.dump({"study_id": "MEMORYLESS-CHECK-2026-10-27", "ks_crit": round(crit, 6), "checks":
checks,
                   "relative_mean_error": {r: round(rel(r), 6) for r in o}, "verdict": verdict},
sys.stdout, indent=1, sort_keys=True)
        print()
```

controls.py

```
        """Instrument controls for the frozen scorer's distribution tests, scored before the
freeze.
        pos_exponential: true exponential samples must pass the memoryless criteria of H1.
        neg_inverse_gaussian_cv1: inverse-Gaussian samples with CV = 1 must fail them (the H2
trap).
        Usage: python3 -I controls.py > controls.json"""
        import json, math, sys, pathlib
        sys.path.insert(0, str(pathlib.Path(__file__).resolve().parent))
        import numpy as np
        from scipy import stats
        from model import describe
        crit = 1.358 / math.sqrt(4000); out = {}
        for name, draw in [("pos_exponential", lambda r: r.exponential(1.0, 4000)),
                           ("neg_inverse_gaussian_cv1", lambda r: stats.invgauss(1.0,
scale=1.0).rvs(4000, random_state=r))]:
            rows = [describe(draw(np.random.default_rng(7000 + i))) for i in range(10)]
            memoryless = sum(d["ks_exp"] <= crit for d in rows) >= 7 and all(0.95 <= d["mrl"][1] <=
1.10 for d in rows)
            out[name] = {"memoryless_criteria_met": bool(memoryless),
                         "ks_exp": [round(d["ks_exp"], 4) for d in rows], "cv": [round(d["cv"], 3)
for d in rows],
                         "mrl_median": [round(d["mrl"][1], 3) for d in rows]}
        out["expected"] = {"pos_exponential": True, "neg_inverse_gaussian_cv1": False}
        out["controls_behave"] = out["pos_exponential"]["memoryless_criteria_met"] and not
out["neg_inverse_gaussian_cv1"]["memoryless_criteria_met"]
        print(json.dumps(out, indent=1, sort_keys=True))
```

run.sh

```
        #!/bin/sh
        # usage: sh run.sh RUNS_DIR   (after the freeze and anchor; generates the holdout seeds
1000-1009 for the first time)
        set -e
        H="$(cd "$(dirname "$0")" && pwd)"; R="$1"; mkdir -p "$R"
        export PYTHONDONTWRITEBYTECODE=1
        python3 -I "$H/run_seeds.py" "$R/outcomes.json" 4000 1000 1001 1002 1003 1004 1005 1006
1007 1008 1009
        python3 -I "$H/score.py" "$R/outcomes.json" > "$R/verdict.json"
```

figure.py

```
        """Figure 1: survival of the inter-cut time in the three regimes, holdout seed 1000
(identical draws to the frozen run).
        Usage: python3 figure.py   (writes ../survival.png)"""
        import numpy as np
        import matplotlib; matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        from model import REGIMES, simulate, start
```

```
        C = {"directed": "#4A7A2C", "drift_diffusion": "#B8893A", "escape": "#A3402B"}
        LAB = {"directed": "directed accumulation", "drift_diffusion": "drift and diffusion
balanced (Péclet 2)", "escape": "escape from a well (Kramers regime)"}
        fig, ax = plt.subplots(figsize=(7.2, 4.4), dpi=200)
        for name, p in REGIMES.items():
            T, _, _ = simulate(**p, n=4000, seed=1000, s0=start(p))
            x = np.sort(T / np.nanmean(T)); surv = 1 - np.arange(1, x.size + 1) / x.size
            cv = np.nanstd(T) / np.nanmean(T)
            ax.semilogy(x[:-1], surv[:-1], color=C[name], lw=2, label=f"{LAB[name]} (CV {cv:.2f})")
        xx = np.linspace(0, 6, 100); ax.semilogy(xx, np.exp(-xx), color="#444", lw=1.2, ls="--",
label="memoryless (exponential)")
        ax.set_xlim(0, 6); ax.set_ylim(1e-3, 1.05)
        ax.set_xlabel("inter-cut time / its mean"); ax.set_ylabel("fraction of occasions still
uncut")
        for sp in ("top", "right"): ax.spines[sp].set_visible(False)
        ax.grid(color="#e6e2d8", lw=0.6); ax.legend(frameon=False, fontsize=8.5)
        fig.tight_layout(); fig.savefig("../survival.png", facecolor="white")
```